

Battleship

Generated by Doxygen 1.16.1

1 Battleship	1
1.1 Projekto nariai	1
1.2 Aprašymas	1
1.3 Projekto struktūra	1
1.3.1 1. Projekto logika	1
1.3.2 2. Vartotojo pasiekiamos funkcijos	1
1.4 Naudojamos technologijos	1
1.5 Darbo apimtis	1
2 Directory Hierarchy	3
2.1 Directories	3
3 Hierarchical Index	5
3.1 Class Hierarchy	5
4 Class Index	7
4.1 Class List	7
5 File Index	9
5.1 File List	9
6 Directory Documentation	11
6.1 include Directory Reference	11
6.2 src Directory Reference	11
6.3 tests Directory Reference	12
7 Class Documentation	13
7.1 AI Class Reference	13
7.1.1 Detailed Description	15
7.1.2 Constructor & Destructor Documentation	15
7.1.2.1 AI()	15
7.1.2.2 ~AI()	15
7.1.3 Member Function Documentation	16
7.1.3.1 attacked()	16
7.1.3.2 fireShot()	16
7.1.3.3 getMyGrid()	16
7.1.3.4 getTargetGrid()	17
7.1.3.5 importBoardFromFile()	17
7.1.3.6 placeShip()	17
7.1.3.7 setupGrid()	17
7.1.3.8 updateTargetGrid()	18
7.1.4 Member Data Documentation	18
7.1.4.1 myBoard	18
7.1.4.2 targetGrid	18

7.2 Board Class Reference	18
7.2.1 Constructor & Destructor Documentation	19
7.2.1.1 Board()	19
7.2.1.2 ~Board()	20
7.2.2 Member Function Documentation	20
7.2.2.1 attack()	20
7.2.2.2 getGrid()	20
7.2.2.3 importFromFile()	21
7.2.2.4 placeShip()	21
7.2.3 Member Data Documentation	21
7.2.3.1 grid	21
7.2.3.2 ships	21
7.3 Coordinates Class Reference	21
7.3.1 Detailed Description	22
7.3.2 Constructor & Destructor Documentation	22
7.3.2.1 Coordinates() [1/2]	22
7.3.2.2 Coordinates() [2/2]	22
7.3.3 Member Data Documentation	22
7.3.3.1 x	22
7.3.3.2 y	23
7.4 Game Class Reference	23
7.4.1 Member Function Documentation	25
7.4.1.1 printBoard()	25
7.4.1.2 printTargetBoard()	25
7.4.1.3 runGame()	26
7.4.1.4 runSetup()	26
7.4.1.5 shipsAlive()	26
7.4.2 Member Data Documentation	26
7.4.2.1 ai	26
7.4.2.2 human	27
7.5 Human Class Reference	27
7.5.1 Detailed Description	29
7.5.2 Constructor & Destructor Documentation	29
7.5.2.1 Human()	29
7.5.2.2 ~Human()	29
7.5.3 Member Function Documentation	30
7.5.3.1 attacked()	30
7.5.3.2 fireShot()	30
7.5.3.3 getMyGrid()	30
7.5.3.4 getTargetGrid()	31
7.5.3.5 importBoardFromFile()	31
7.5.3.6 placeShip()	31

7.5.3.7 updateTargetGrid()	32
7.5.4 Member Data Documentation	32
7.5.4.1 myBoard	32
7.5.4.2 targetGrid	32
7.6 Player Class Reference	32
7.6.1 Detailed Description	35
7.6.2 Constructor & Destructor Documentation	35
7.6.2.1 Player()	35
7.6.2.2 ~Player()	35
7.6.3 Member Function Documentation	36
7.6.3.1 attacked()	36
7.6.3.2 fireShot()	36
7.6.3.3 getMyGrid()	36
7.6.3.4 getTargetGrid()	37
7.6.3.5 importBoardFromFile()	37
7.6.3.6 placeShip()	37
7.6.3.7 updateTargetGrid()	38
7.6.4 Member Data Documentation	38
7.6.4.1 myBoard	38
7.6.4.2 targetGrid	38
7.7 Ship Class Reference	38
7.7.1 Constructor & Destructor Documentation	40
7.7.1.1 Ship() [1/2]	40
7.7.1.2 ~Ship()	40
7.7.1.3 Ship() [2/2]	40
7.7.2 Member Function Documentation	40
7.7.2.1 checkHit()	40
7.7.2.2 getIsSunk()	41
7.7.3 Member Data Documentation	41
7.7.3.1 end	41
7.7.3.2 hitCount	41
7.7.3.3 isSunk	41
7.7.3.4 start	41
8 File Documentation	43
8.1 include/AI.h File Reference	43
8.1.1 Detailed Description	44
8.2 AI.h	44
8.3 include/Board.h File Reference	44
8.3.1 Detailed Description	45
8.4 Board.h	45
8.5 include/Coordinates.h File Reference	46

8.5.1 Detailed Description	46
8.6 Coordinates.h	47
8.7 include/Game.h File Reference	47
8.7.1 Detailed Description	48
8.8 Game.h	48
8.9 include/Human.h File Reference	49
8.9.1 Detailed Description	50
8.10 Human.h	50
8.11 include/Player.h File Reference	50
8.11.1 Detailed Description	51
8.12 Player.h	51
8.13 include/Ship.h File Reference	52
8.13.1 Detailed Description	52
8.14 Ship.h	53
8.15 README.md File Reference	53
8.16 src/AI.cpp File Reference	53
8.16.1 Detailed Description	54
8.16.2 Variable Documentation	54
8.16.2.1 SHIP_COUNT	54
8.16.2.2 SHIP_SIZES	55
8.17 src/Board.cpp File Reference	55
8.17.1 Detailed Description	55
8.18 src/Game.cpp File Reference	55
8.18.1 Detailed Description	56
8.19 src/Human.cpp File Reference	56
8.19.1 Detailed Description	57
8.20 src/main.cpp File Reference	57
8.20.1 Detailed Description	58
8.20.2 Function Documentation	58
8.20.2.1 main()	58
8.21 src/Player.cpp File Reference	59
8.21.1 Detailed Description	59
8.22 src/Ship.cpp File Reference	59
8.22.1 Detailed Description	60
8.23 tests/AITest.cpp File Reference	61
8.23.1 Function Documentation	61
8.23.1.1 testAI()	61
8.24 tests/AITest.h File Reference	62
8.24.1 Function Documentation	63
8.24.1.1 testAI()	63
8.25 AITest.h	63
8.26 tests/BoardTest.cpp File Reference	63

8.26.1 Function Documentation	64
8.26.1.1 testBoard()	64
8.27 tests/BoardTest.h File Reference	64
8.27.1 Function Documentation	65
8.27.1.1 testBoard()	65
8.28 BoardTest.h	65
8.29 tests/HumanTest.cpp File Reference	65
8.29.1 Function Documentation	66
8.29.1.1 testHuman()	66
8.30 tests/HumanTest.h File Reference	67
8.30.1 Function Documentation	68
8.30.1.1 testHuman()	68
8.31 HumanTest.h	68
8.32 tests/ShipTest.cpp File Reference	68
8.32.1 Function Documentation	69
8.32.1.1 testShip()	69
8.33 tests/ShipTest.h File Reference	69
8.33.1 Function Documentation	70
8.33.1.1 testShip()	70
8.34 ShipTest.h	70
8.35 tests/test.cpp File Reference	70
8.35.1 Function Documentation	71
8.35.1.1 main()	71
Index	73

Chapter 1

Battleship

1.1 Projekto nariai

- Justas Kurtinaitis (builder, designer)
- Edgar Dainarovič (architect, critic)
- Kamil Bužinski (facilitator, exhibitor).

1.2 Aprašymas

Šio projekto tikslas - sukurti Battleship žaidimo kopiją.

1.3 Projekto struktūra

1.3.1 1. Projekto logika

1.3.2 2. Vartotojo pasiekiamos funkcijos

1.4 Naudojamos technologijos

Grafiniam interfeisui:

- [Raylib](#)

Bendradarbiavimui:

- [GitHub](#)
- [Discord](#)

1.5 Darbo apimtis

Darbas buvo paskirstytas polygiai.

Chapter 2

Directory Hierarchy

2.1 Directories

include	11
Al.h	43
Board.h	44
Coordinates.h	46
Game.h	47
Human.h	49
Player.h	50
Ship.h	52
src	11
Al.cpp	53
Board.cpp	55
Game.cpp	55
Human.cpp	56
main.cpp	57
Player.cpp	59
Ship.cpp	59
tests	12
AlTest.cpp	61
AlTest.h	62
BoardTest.cpp	63
BoardTest.h	64
HumanTest.cpp	65
HumanTest.h	67
ShipTest.cpp	68
ShipTest.h	69
test.cpp	70

Chapter 3

Hierarchical Index

3.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Board	18
Coordinates	21
Game	23
Player	32
AI	13
Human	27
Ship	38

Chapter 4

Class Index

4.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

AI		
	Represents a computer-controlled player in a game of Battleship	13
Board		18
Coordinates		
	Represents a point in 2D space with x and y coordinates	21
Game		23
Human		
	Represents a human-controlled player in a game of Battleship	27
Player		
	Abstract base class representing a player in a game of Battleship	32
Ship		38

Chapter 5

File Index

5.1 File List

Here is a list of all files with brief descriptions:

include/ AI.h	Defines the AI class, which represents a computer-controlled player in a game of Battleship . . .	43
include/ Board.h	Defines the Board class, which represents the game board in a game of Battleship	44
include/ Coordinates.h	Defines the Coordinates struct, which represents a point in 2D space with x and y coordinates	46
include/ Game.h	Defines The game class, which allows users to interact with the game	47
include/ Human.h	Defines the Human class, which represents a human player in a game of Battleship	49
include/ Player.h	Defines the abstract Player class, which serves as the base class for Human and AI players in a game of Battleship	50
include/ Ship.h	Defines the Ship class, which represents a ship in a game of Battleship	52
src/ AI.cpp	Implements the AI class	53
src/ Board.cpp	Implements the Board class, which represents the game board in a game of Battleship	55
src/ Game.cpp	Implementation of Game class	55
src/ Human.cpp	Implements the Human class	56
src/ main.cpp	Main game file that runs the game	57
src/ Player.cpp	Implements the Player class, the abstract base for Human and AI players in a game of Battleship	59
src/ Ship.cpp	Implements the Ship class, which represents a ship in a game of Battleship	59
tests/ AITest.cpp		61
tests/ AITest.h		62
tests/ BoardTest.cpp		63
tests/ BoardTest.h		64
tests/ HumanTest.cpp		65
tests/ HumanTest.h		67
tests/ ShipTest.cpp		68
tests/ ShipTest.h		69
tests/ test.cpp		70

Chapter 6

Directory Documentation

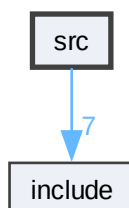
6.1 include Directory Reference

Files

- file [AI.h](#)
Defines the [AI](#) class, which represents a computer-controlled player in a game of Battleship.
- file [Board.h](#)
Defines the [Board](#) class, which represents the game board in a game of Battleship.
- file [Coordinates.h](#)
Defines the [Coordinates](#) struct, which represents a point in 2D space with x and y coordinates.
- file [Game.h](#)
Defines The game class, which allows users to interact with the game.
- file [Human.h](#)
Defines the [Human](#) class, which represents a human player in a game of Battleship.
- file [Player.h](#)
Defines the abstract [Player](#) class, which serves as the base class for [Human](#) and [AI](#) players in a game of Battleship.
- file [Ship.h](#)
Defines the [Ship](#) class, which represents a ship in a game of Battleship.

6.2 src Directory Reference

Directory dependency graph for src:



Files

- file [AI.cpp](#)
Implements the [AI](#) class.
- file [Board.cpp](#)
Implements the [Board](#) class, which represents the game board in a game of Battleship.
- file [Game.cpp](#)
Implementation of [Game](#) class.
- file [Human.cpp](#)
Implements the [Human](#) class.
- file [main.cpp](#)
main game file that runs the game.
- file [Player.cpp](#)
Implements the [Player](#) class, the abstract base for [Human](#) and [AI](#) players in a game of Battleship.
- file [Ship.cpp](#)
Implements the [Ship](#) class, which represents a ship in a game of Battleship.

6.3 tests Directory Reference

Directory dependency graph for tests:



Files

- file [AITest.cpp](#)
- file [AITest.h](#)
- file [BoardTest.cpp](#)
- file [BoardTest.h](#)
- file [HumanTest.cpp](#)
- file [HumanTest.h](#)
- file [ShipTest.cpp](#)
- file [ShipTest.h](#)
- file [test.cpp](#)

Chapter 7

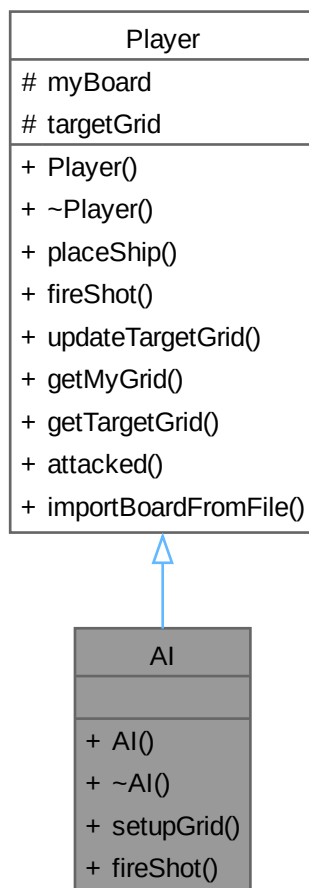
Class Documentation

7.1 AI Class Reference

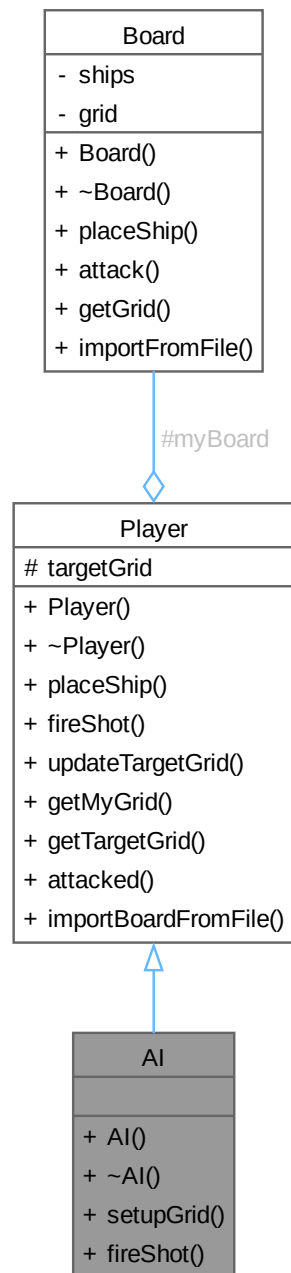
Represents a computer-controlled player in a game of Battleship.

```
#include <AI.h>
```

Inheritance diagram for AI:



Collaboration diagram for AI:



Public Member Functions

- `AI ()`
Default constructor. Delegates initialisation to `Player()`.
- `~AI ()=default`
Default destructor.
- void `setupGrid ()`

- Automatically places all required ships on the AI's own board.*
 - `Coordinates fireShot ()` override
 - Automatically selects a target cell on the opponent's board.*
 - `bool placeShip (Ship ship)`
 - Attempts to place a ship on the player's own board.*
 - `void updateTargetGrid (Coordinates coords, bool hit)`
 - Records the outcome of a shot on the target grid.*
 - `std::vector< std::vector< int > > getMyGrid () const`
 - Returns a pointer to the player's own 10×10 board grid.*
 - `std::vector< std::vector< int > > getTargetGrid () const`
 - Returns a pointer to the player's 10×10 target-tracking grid.*
 - `bool attacked (Coordinates attack)`
 - a wrapper for Board::attack, to be called if attacking this player*
 - `void importBoardFromFile (const std::string &filename)`
 - Imports the player's board configuration from a file.*

Protected Attributes

- `Board myBoard`
- `std::vector< std::vector< int > > targetGrid`

7.1.1 Detailed Description

Represents a computer-controlled player in a game of Battleship.

Extends `Player` with:

- `setupGrid()` : automatically places all ships on the AI's own board using random-but-valid positions.
- `fireShot()` : automatically selects an un-fired target cell. The base implementation uses a random strategy; the game logic layer may call more sophisticated selection.

7.1.2 Constructor & Destructor Documentation

7.1.2.1 AI()

```
AI::AI ()
```

Default constructor. Delegates initialisation to `Player()`.

7.1.2.2 ~AI()

```
AI::~AI () [default]
```

Default destructor.

7.1.3 Member Function Documentation

7.1.3.1 attacked()

```
bool Player::attacked (
    Coordinates attack) [inherited]
```

a wrapper for [Board::attack](#), to be called if attacking this player

wrapper for board::attack

Parameters

<i>attack</i>	coordinates to be attacked
---------------	----------------------------

Returns

true
false

Parameters

<i>attack</i>	The coordinates of the attack.
---------------	--------------------------------

Returns

true if the attack hits a ship, false if it misses.

7.1.3.2 fireShot()

```
Coordinates AI::fireShot () [override], [virtual]
```

Automatically selects a target cell on the opponent's board.

Picks a cell whose targetGrid value is 0 (not yet attacked). The selection strategy (random vs. hunt/target) is determined by the logic implemented in the body of this method.

Returns

[Coordinates](#) The cell the [AI](#) chooses to attack.

Implements [Player](#).

7.1.3.3 getMyGrid()

```
std::vector< std::vector< int > > Player::getMyGrid () const [inherited]
```

Returns a pointer to the player's own 10×10 board grid.

Grid encoding: 0 = empty, 1 = ship, 2 = hit, 3 = miss.

Returns

vector<vector<int>> internal 2-D array of myBoard.

7.1.3.4 `getTargetGrid()`

```
std::vector< std::vector< int > > Player::getTargetGrid () const [inherited]
```

Returns a pointer to the player's 10×10 target-tracking grid.

Grid encoding: 0 = unknown, 2 = hit, 3 = miss.

Returns

`vector<vector<int>>` internal 2-D target grid array.

7.1.3.5 `importBoardFromFile()`

```
void Player::importBoardFromFile (
    const std::string & filename) [inline], [inherited]
```

Imports the player's board configuration from a file.

Parameters

<i>filename</i>	The name of the file containing the board configuration (ship placements).
-----------------	--

7.1.3.6 `placeShip()`

```
bool Player::placeShip (
    Ship ship) [inherited]
```

Attempts to place a ship on the player's own board.

Delegates to [Board::placeShip](#), which validates bounds and overlap.

Parameters

<i>ship</i>	The ship to place (start/end coordinates must already be set).
-------------	--

Returns

true if the ship was placed successfully.

false if the placement is invalid (out of bounds or overlapping).

7.1.3.7 `setupGrid()`

```
void AI::setupGrid ()
```

Automatically places all required ships on the AI's own board.

Generates random start coordinates and orientation for each ship, retrying until [Board::placeShip\(\)](#) accepts the placement. Called once during the Setup phase (see activity diagram).

7.1.3.8 updateTargetGrid()

```
void Player::updateTargetGrid (
    Coordinates coords,
    bool hit) [inherited]
```

Records the outcome of a shot on the target grid.

Should be called after the opponent's [Board::attack\(\)](#) returns so that the player's local view stays in sync with the opponent's board state.

Parameters

<i>coords</i>	The cell that was attacked.
<i>hit</i>	true if the shot hit a ship, false if it missed.

7.1.4 Member Data Documentation

7.1.4.1 myBoard

```
Board Player::myBoard [protected], [inherited]
```

The player's own board, containing their ships.

7.1.4.2 targetGrid

```
std::vector<std::vector<int> > Player::targetGrid [protected], [inherited]
```

Local tracking grid for shots fired at the opponent. 0 = unknown, 2 = hit, 3 = miss (mirrors [Board](#) grid encoding).

The documentation for this class was generated from the following files:

- include/[AI.h](#)
- src/[AI.cpp](#)

7.2 Board Class Reference

```
#include <Board.h>
```

Collaboration diagram for Board:

Board
- ships - grid
+ Board() + ~Board() + placeShip() + attack() + getGrid() + importFromFile()

Public Member Functions

- [Board](#) ()
Default constructor that initializes the board with default values.
- [~Board](#) ()=default
Places a ship on the board at the specified coordinates.
- bool [placeShip](#) ([Ship](#) ship)
- bool [attack](#) ([Coordinates](#) attack)
Attacks the specified coordinates on the board.
- std::vector< std::vector< int > > [getGrid](#) () const
Returns a pointer to the 2D array representing the game board.
- void [importFromFile](#) (const std::string &filename)
Imports the game board configuration from a file (file format - coordinates of the ships).

Private Attributes

- std::vector< [Ship](#) > [ships](#)
- std::vector< std::vector< int > > [grid](#)

7.2.1 Constructor & Destructor Documentation

7.2.1.1 Board()

```
Board::Board ()
```

Default constructor that initializes the board with default values.

7.2.1.2 ~Board()

```
Board::~~Board () [default]
```

Places a ship on the board at the specified coordinates.

Parameters

<i>ship</i>	The ship to be placed on the board.
-------------	-------------------------------------

Returns

true if the ship was successfully placed, false if the placement is invalid (e.g., out of bounds or overlapping with another ship).

Destructor for the [Board](#) class. arrays and vectors will automatically clean up their memory, so no explicit cleanup is necessary in this destructor.

7.2.2 Member Function Documentation

7.2.2.1 attack()

```
bool Board::attack (
    Coordinates attack)
```

Attacks the specified coordinates on the board.

Parameters

<i>attack</i>	The coordinates of the attack.
---------------	--------------------------------

Returns

true if the attack hits a ship, false if it misses.

7.2.2.2 getGrid()

```
std::vector< std::vector< int > > Board::getGrid () const [inline]
```

Returns a pointer to the 2D array representing the game board.

Returns

A pointer to the 2D array representing the game board

7.2.2.3 importFromFile()

```
void Board::importFromFile (
    const std::string & filename)
```

Imports the game board configuration from a file (file format - coordinates of the ships).

Parameters

<i>filename</i>	The name of the file containing the game board configuration.
-----------------	---

7.2.2.4 placeShip()

```
bool Board::placeShip (
    Ship ship)
```

7.2.3 Member Data Documentation

7.2.3.1 grid

```
std::vector<std::vector<int> > Board::grid [private]
```

A 2D array representing the game board, where 0 indicates an empty cell, 1 indicates a cell occupied by a ship, 2 indicates hit, and 3 indicates a miss.

7.2.3.2 ships

```
std::vector<Ship> Board::ships [private]
```

An array of ships on the board.

The documentation for this class was generated from the following files:

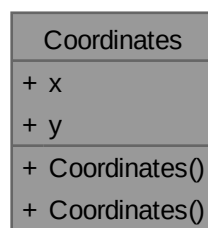
- include/[Board.h](#)
- src/[Board.cpp](#)

7.3 Coordinates Class Reference

Represents a point in 2D space with x and y coordinates.

```
#include <Coordinates.h>
```

Collaboration diagram for Coordinates:



Public Member Functions

- [Coordinates](#) ()
Default constructor that initializes the coordinates to (0, 0).
- [Coordinates](#) (int [x](#), int [y](#))
Parameterized constructor that initializes the coordinates to the specified values.

Public Attributes

- int [x](#)
- int [y](#)

7.3.1 Detailed Description

Represents a point in 2D space with x and y coordinates.

7.3.2 Constructor & Destructor Documentation

7.3.2.1 [Coordinates](#)() [1/2]

```
Coordinates::Coordinates () [inline]
```

Default constructor that initializes the coordinates to (0, 0).

7.3.2.2 [Coordinates](#)() [2/2]

```
Coordinates::Coordinates (  
    int x,  
    int y) [inline]
```

Parameterized constructor that initializes the coordinates to the specified values.

Parameters

x	
y	

7.3.3 Member Data Documentation

7.3.3.1 [x](#)

```
int Coordinates::x
```

7.3.3.2 y

```
int Coordinates::y
```

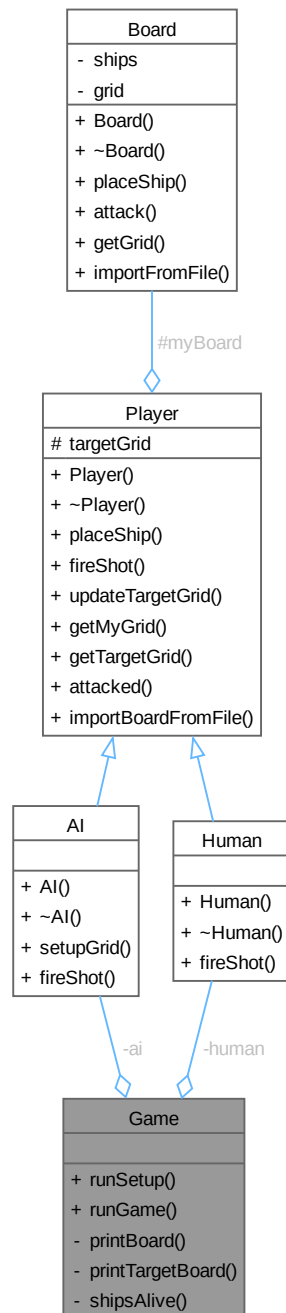
The documentation for this class was generated from the following file:

- include/[Coordinates.h](#)

7.4 Game Class Reference

```
#include <Game.h>
```

Collaboration diagram for Game:



Public Member Functions

- void `runSetup()`
runs thru ai and human board setup
- void `runGame()`
runs the game loop

Private Member Functions

- void `printBoard` (`std::vector< std::vector< int > > board`)
prints out the given board parameter
- void `printTargetBoard` (`std::vector< std::vector< int > > board`)
checks if there are alive ships on a given board
- bool `shipsAlive` (`std::vector< std::vector< int > > board`)
checks if there are alive ships on a given board

Private Attributes

- `AI ai`
The AI player.
- `Human human`
The human player.

7.4.1 Member Function Documentation

7.4.1.1 `printBoard()`

```
void Game::printBoard (  
    std::vector< std::vector< int > > board) [private]
```

prints out the given board parameter

Parameters

<i>board</i>	Players board
--------------	---------------

7.4.1.2 `printTargetBoard()`

```
void Game::printTargetBoard (  
    std::vector< std::vector< int > > board) [private]
```

checks if there are alive ships on a given board

Parameters

<i>board</i>	Players board
--------------	---------------

Returns

true there are alive ships
false no alive ships are left

prints out the given board parameter, but hides the ships

Parameters

<i>board</i>	
--------------	--

7.4.1.3 runGame()

```
void Game::runGame ()
```

runs the game loop

7.4.1.4 runSetup()

```
void Game::runSetup ()
```

runs thru ai and human board setup

7.4.1.5 shipsAlive()

```
bool Game::shipsAlive (  
    std::vector< std::vector< int > > board) [private]
```

checks if there are alive ships on a given board

Parameters

<i>board</i>	
--------------	--

Returns

true there are alive ships, else false

7.4.2 Member Data Documentation**7.4.2.1 ai**

```
AI Game::ai [private]
```

The AI player.

7.4.2.2 human

```
Human Game::human [private]
```

The human player.

The documentation for this class was generated from the following files:

- include/[Game.h](#)
- src/[Game.cpp](#)

7.5 Human Class Reference

Represents a human-controlled player in a game of Battleship.

```
#include <Human.h>
```

Inheritance diagram for Human:



Collaboration diagram for Human:



Public Member Functions

- [Human](#) ()
Default constructor. Delegates initialisation to [Player](#)().
- [~Human](#) ()=default
Default destructor.
- [Coordinates fireShot](#) () override

- Prompts the human player to enter attack coordinates.*
- bool [placeShip](#) ([Ship](#) ship)
Attempts to place a ship on the player's own board.
- void [updateTargetGrid](#) ([Coordinates](#) coords, bool hit)
Records the outcome of a shot on the target grid.
- std::vector< std::vector< int > > [getMyGrid](#) () const
Returns a pointer to the player's own 10×10 board grid.
- std::vector< std::vector< int > > [getTargetGrid](#) () const
Returns a pointer to the player's 10×10 target-tracking grid.
- bool [attacked](#) ([Coordinates](#) attack)
a wrapper for [Board::attack](#), to be called if attacking this player
- void [importBoardFromFile](#) (const std::string &filename)
Imports the player's board configuration from a file.

Protected Attributes

- [Board](#) myBoard
- std::vector< std::vector< int > > [targetGrid](#)

7.5.1 Detailed Description

Represents a human-controlled player in a game of Battleship.

Extends [Player](#) with a [fireShot\(\)](#) implementation that obtains the target coordinates from user input (keyboard). [Ship](#) placement is interactive: the game loop is responsible for reading the desired position and rotation from the user and calling [placeShip\(\)](#) accordingly.

7.5.2 Constructor & Destructor Documentation

7.5.2.1 Human()

```
Human::Human ()
```

Default constructor. Delegates initialisation to [Player\(\)](#).

7.5.2.2 ~Human()

```
Human::~Human () [default]
```

Default destructor.

7.5.3 Member Function Documentation

7.5.3.1 attacked()

```
bool Player::attacked (
    Coordinates attack) [inherited]
```

a wrapper for [Board::attack](#), to be called if attacking this player

wrapper for board::attack

Parameters

<i>attack</i>	coordinates to be attacked
---------------	----------------------------

Returns

true
false

Parameters

<i>attack</i>	The coordinates of the attack.
---------------	--------------------------------

Returns

true if the attack hits a ship, false if it misses.

7.5.3.2 fireShot()

```
Coordinates Human::fireShot () [override], [virtual]
```

Prompts the human player to enter attack coordinates.

Reads a target cell from standard input, validates that the cell has not already been fired upon (targetGrid value is 0), and returns the chosen coordinates. Re-prompts on invalid input.

Returns

[Coordinates](#) The cell the human player wishes to attack.

Implements [Player](#).

7.5.3.3 getMyGrid()

```
std::vector< std::vector< int > > Player::getMyGrid () const [inherited]
```

Returns a pointer to the player's own 10×10 board grid.

Grid encoding: 0 = empty, 1 = ship, 2 = hit, 3 = miss.

Returns

vector<vector<int>> internal 2-D array of myBoard.

7.5.3.4 `getTargetGrid()`

```
std::vector< std::vector< int > > Player::getTargetGrid () const [inherited]
```

Returns a pointer to the player's 10×10 target-tracking grid.

Grid encoding: 0 = unknown, 2 = hit, 3 = miss.

Returns

`vector<vector<int>>` internal 2-D target grid array.

7.5.3.5 `importBoardFromFile()`

```
void Player::importBoardFromFile (
    const std::string & filename) [inline], [inherited]
```

Imports the player's board configuration from a file.

Parameters

<i>filename</i>	The name of the file containing the board configuration (ship placements).
-----------------	--

7.5.3.6 `placeShip()`

```
bool Player::placeShip (
    Ship ship) [inherited]
```

Attempts to place a ship on the player's own board.

Delegates to [Board::placeShip](#), which validates bounds and overlap.

Parameters

<i>ship</i>	The ship to place (start/end coordinates must already be set).
-------------	--

Returns

true if the ship was placed successfully.

false if the placement is invalid (out of bounds or overlapping).

7.5.3.7 updateTargetGrid()

```
void Player::updateTargetGrid (
    Coordinates coords,
    bool hit) [inherited]
```

Records the outcome of a shot on the target grid.

Should be called after the opponent's [Board::attack\(\)](#) returns so that the player's local view stays in sync with the opponent's board state.

Parameters

<i>coords</i>	The cell that was attacked.
<i>hit</i>	true if the shot hit a ship, false if it missed.

7.5.4 Member Data Documentation

7.5.4.1 myBoard

```
Board Player::myBoard [protected], [inherited]
```

The player's own board, containing their ships.

7.5.4.2 targetGrid

```
std::vector<std::vector<int> > Player::targetGrid [protected], [inherited]
```

Local tracking grid for shots fired at the opponent. 0 = unknown, 2 = hit, 3 = miss (mirrors [Board](#) grid encoding).

The documentation for this class was generated from the following files:

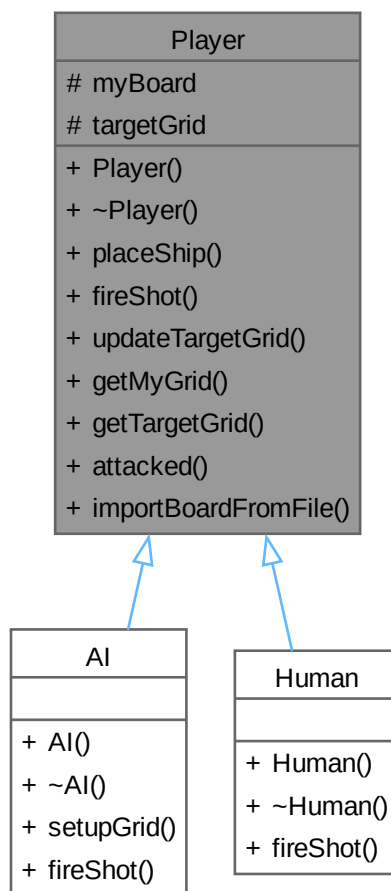
- include/[Human.h](#)
- src/[Human.cpp](#)

7.6 Player Class Reference

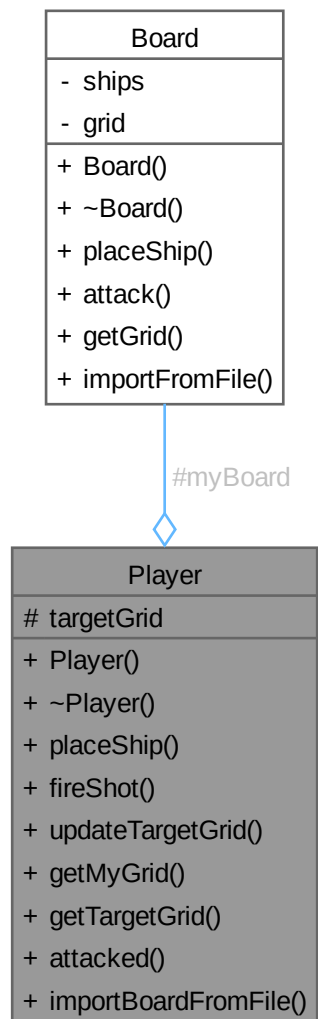
Abstract base class representing a player in a game of Battleship.

```
#include <Player.h>
```


Inheritance diagram for Player:



Collaboration diagram for Player:



Public Member Functions

- `Player ()`
Default constructor. Initialises the target grid to all zeros (no shots fired yet).
- `virtual ~Player ()=default`
Virtual destructor. Derived classes clean up their own resources.
- `bool placeShip (Ship ship)`
Attempts to place a ship on the player's own board.
- `virtual Coordinates fireShot ()=0`
Selects the coordinates for the next attack on the opponent's board.
- `void updateTargetGrid (Coordinates coords, bool hit)`
Records the outcome of a shot on the target grid.
- `std::vector< std::vector< int > > getMyGrid () const`

- Returns a pointer to the player's own 10×10 board grid.*
- `std::vector< std::vector< int > > getTargetGrid () const`
Returns a pointer to the player's 10×10 target-tracking grid.
- `bool attacked (Coordinates attack)`
a wrapper for [Board::attack](#), to be called if attacking this player
- `void importBoardFromFile (const std::string &filename)`
Imports the player's board configuration from a file.

Protected Attributes

- `Board myBoard`
- `std::vector< std::vector< int > > targetGrid`

7.6.1 Detailed Description

Abstract base class representing a player in a game of Battleship.

Holds the player's own board (where their ships reside) and a target grid (a local record of shots fired at the opponent's board). Concrete subclasses must implement [fireShot\(\)](#), which differs between a [Human](#) (user input) and an [AI](#) (automated selection).

7.6.2 Constructor & Destructor Documentation

7.6.2.1 `Player()`

```
Player::Player ()
```

Default constructor. Initialises the target grid to all zeros (no shots fired yet).

7.6.2.2 `~Player()`

```
virtual Player::~~Player () [virtual], [default]
```

Virtual destructor. Derived classes clean up their own resources.

7.6.3 Member Function Documentation

7.6.3.1 attacked()

```
bool Player::attacked (
    Coordinates attack)
```

a wrapper for [Board::attack](#), to be called if attacking this player

wrapper for board::attack

Parameters

<i>attack</i>	coordinates to be attacked
---------------	----------------------------

Returns

true
false

Parameters

<i>attack</i>	The coordinates of the attack.
---------------	--------------------------------

Returns

true if the attack hits a ship, false if it misses.

7.6.3.2 fireShot()

```
virtual Coordinates Player::fireShot () [pure virtual]
```

Selects the coordinates for the next attack on the opponent's board.

Pure virtual — [Human](#) reads from user input; [AI](#) computes a target automatically.

Returns

[Coordinates](#) The cell to attack on the opponent's board.

Implemented in [AI](#), and [Human](#).

7.6.3.3 getMyGrid()

```
std::vector< std::vector< int > > Player::getMyGrid () const
```

Returns a pointer to the player's own 10×10 board grid.

Grid encoding: 0 = empty, 1 = ship, 2 = hit, 3 = miss.

Returns

vector<vector<int>> internal 2-D array of myBoard.

7.6.3.4 `getTargetGrid()`

```
std::vector< std::vector< int > > Player::getTargetGrid () const
```

Returns a pointer to the player's 10×10 target-tracking grid.

Grid encoding: 0 = unknown, 2 = hit, 3 = miss.

Returns

`vector<vector<int>>` internal 2-D target grid array.

7.6.3.5 `importBoardFromFile()`

```
void Player::importBoardFromFile (  
    const std::string & filename) [inline]
```

Imports the player's board configuration from a file.

Parameters

<i>filename</i>	The name of the file containing the board configuration (ship placements).
-----------------	--

7.6.3.6 `placeShip()`

```
bool Player::placeShip (  
    Ship ship)
```

Attempts to place a ship on the player's own board.

Delegates to `Board::placeShip`, which validates bounds and overlap.

Parameters

<i>ship</i>	The ship to place (start/end coordinates must already be set).
-------------	--

Returns

true if the ship was placed successfully.

false if the placement is invalid (out of bounds or overlapping).

7.6.3.7 updateTargetGrid()

```
void Player::updateTargetGrid (
    Coordinates coords,
    bool hit)
```

Records the outcome of a shot on the target grid.

Should be called after the opponent's [Board::attack\(\)](#) returns so that the player's local view stays in sync with the opponent's board state.

Parameters

<i>coords</i>	The cell that was attacked.
<i>hit</i>	true if the shot hit a ship, false if it missed.

7.6.4 Member Data Documentation

7.6.4.1 myBoard

```
Board Player::myBoard [protected]
```

The player's own board, containing their ships.

7.6.4.2 targetGrid

```
std::vector<std::vector<int> > Player::targetGrid [protected]
```

Local tracking grid for shots fired at the opponent. 0 = unknown, 2 = hit, 3 = miss (mirrors [Board](#) grid encoding).

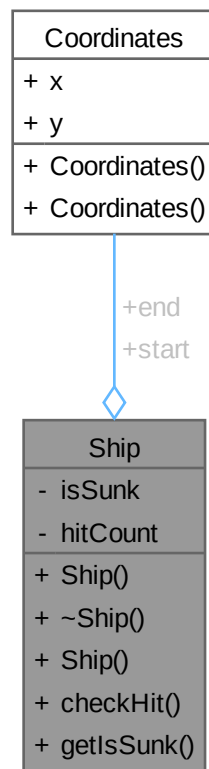
The documentation for this class was generated from the following files:

- include/[Player.h](#)
- src/[Player.cpp](#)

7.7 Ship Class Reference

```
#include <Ship.h>
```

Collaboration diagram for Ship:



Public Member Functions

- [Ship \(\)](#)
Default constructor that initializes the ship with default values.
- [~Ship \(\)](#)=default
Destructor for the [Ship](#) class. Since we are not dynamically allocating any memory in this class, we can use the default destructor provided by the compiler.
- [Ship \(Coordinates start, Coordinates end\)](#)
Parameterized constructor that initializes the ship with the specified starting and ending coordinates.
- bool [checkHit \(Coordinates attack\)](#)
Checks if the given attack coordinates hit the ship.
- bool [getIsSunk \(\)](#)
Returns whether the ship has been sunk.

Public Attributes

- [Coordinates start](#)
- [Coordinates end](#)

Private Attributes

- bool `isSunk`
- int `hitCount`

7.7.1 Constructor & Destructor Documentation

7.7.1.1 `Ship()` [1/2]

```
Ship::Ship () [inline]
```

Default constructor that initializes the ship with default values.

7.7.1.2 `~Ship()`

```
Ship::~Ship () [default]
```

Destructor for the `Ship` class. Since we are not dynamically allocating any memory in this class, we can use the default destructor provided by the compiler.

7.7.1.3 `Ship()` [2/2]

```
Ship::Ship (
    Coordinates start,
    Coordinates end)
```

Parameterized constructor that initializes the ship with the specified starting and ending coordinates.

Parameters

<code>start</code>	
<code>end</code>	

7.7.2 Member Function Documentation

7.7.2.1 `checkHit()`

```
bool Ship::checkHit (
    Coordinates attack)
```

Checks if the given attack coordinates hit the ship.

Parameters

<code>attack</code>	The coordinates of the attack.
---------------------	--------------------------------

Returns

true if the attack hits the ship, false otherwise.

7.7.2.2 `getIsSunk()`

```
bool Ship::getIsSunk ()
```

Returns whether the ship has been sunk.

Returns

true if the ship has been sunk, false otherwise.

7.7.3 Member Data Documentation

7.7.3.1 `end`

```
Coordinates Ship::end
```

The ending coordinates of the ship.

7.7.3.2 `hitCount`

```
int Ship::hitCount [private]
```

The number of hits the ship has taken.

7.7.3.3 `isSunk`

```
bool Ship::isSunk [private]
```

A flag indicating whether the ship has been sunk.

7.7.3.4 `start`

```
Coordinates Ship::start
```

The starting coordinates of the ship.

The documentation for this class was generated from the following files:

- `include/Ship.h`
- `src/Ship.cpp`

Chapter 8

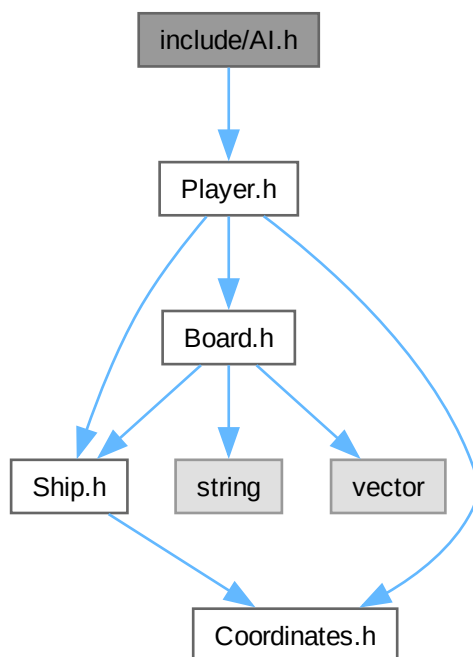
File Documentation

8.1 include/AI.h File Reference

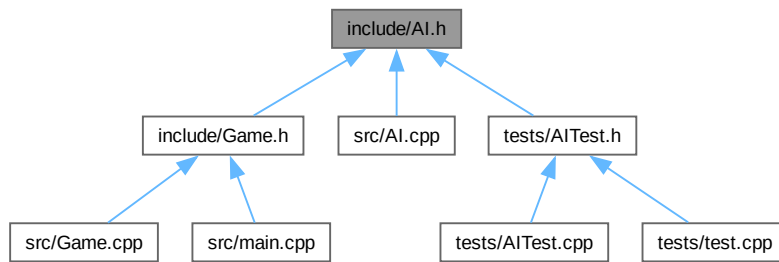
Defines the [AI](#) class, which represents a computer-controlled player in a game of Battleship.

```
#include "Player.h"
```

Include dependency graph for AI.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [AI](#)

Represents a computer-controlled player in a game of Battleship.

8.1.1 Detailed Description

Defines the [AI](#) class, which represents a computer-controlled player in a game of Battleship.

8.2 AI.h

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include "Player.h"
00010
00022 class AI : public Player {
00023 public:
00027     AI();
00028
00032     ~AI() = default;
00033
00041     void setupGrid();
00042
00052     Coordinates fireShot() override;
00053 };
  
```

8.3 include/Board.h File Reference

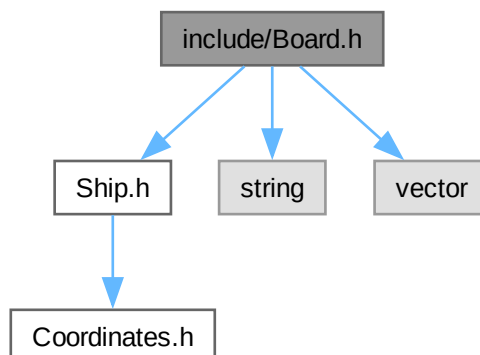
Defines the [Board](#) class, which represents the game board in a game of Battleship.

```

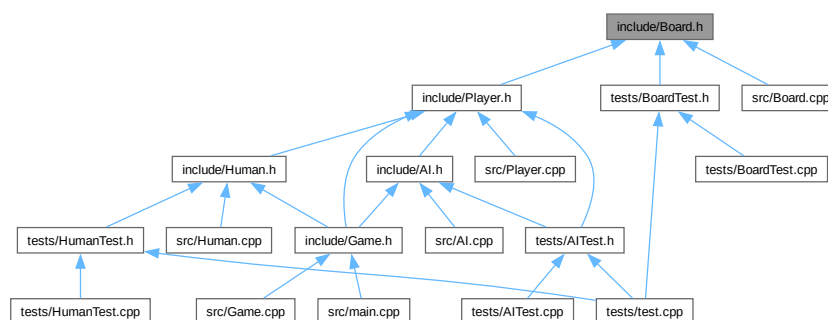
#include "Ship.h"
#include <string>
  
```

```
#include <vector>
```

Include dependency graph for Board.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Board](#)

8.3.1 Detailed Description

Defines the [Board](#) class, which represents the game board in a game of Battleship.

8.4 Board.h

[Go to the documentation of this file.](#)

00001

```

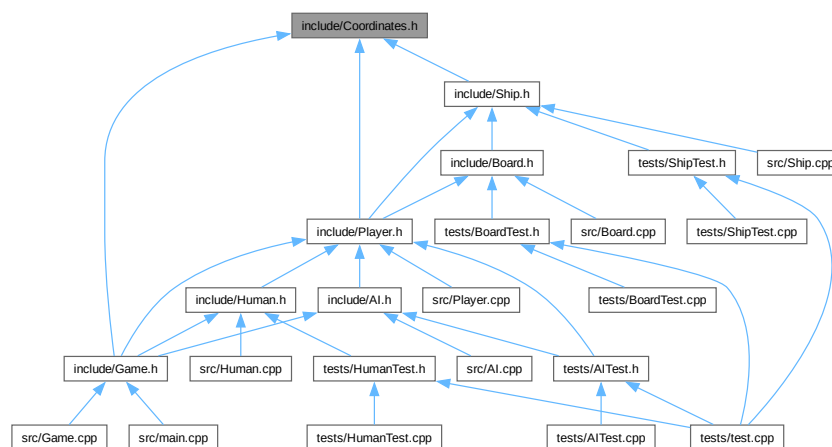
00006
00007 #pragma once
00008
00009 #include "Ship.h"
00010 #include <string>
00011 #include <vector>
00012
00013 /****
00014 * @class Board
00015 * @brief Represents the game board in a game of Battleship.
00016 */
00017 class Board {
00018 private:
00019     std::vector<Ship> ships;
00020     std::vector<std::vector<int>>
00021         grid;
00024 public:
00028     Board();
00041     ~Board() = default;
00042     bool placeShip(Ship ship);
00048     bool attack(Coordinates attack);
00049
00055     std::vector<std::vector<int>> getGrid() const { return grid; }
00056
00064     void importFromFile(const std::string &filename);
00065 };

```

8.5 include/Coordinates.h File Reference

Defines the [Coordinates](#) struct, which represents a point in 2D space with x and y coordinates.

This graph shows which files directly or indirectly include this file:



Classes

- class [Coordinates](#)

Represents a point in 2D space with x and y coordinates.

8.5.1 Detailed Description

Defines the [Coordinates](#) struct, which represents a point in 2D space with x and y coordinates.

8.6 Coordinates.h

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00014 struct Coordinates {
00015     int x;
00016     int y;
00017
00021     Coordinates() : x(0), y(0) {}
00029     Coordinates(int x, int y) : x(x), y(y) {}
00030 };

```

8.7 include/Game.h File Reference

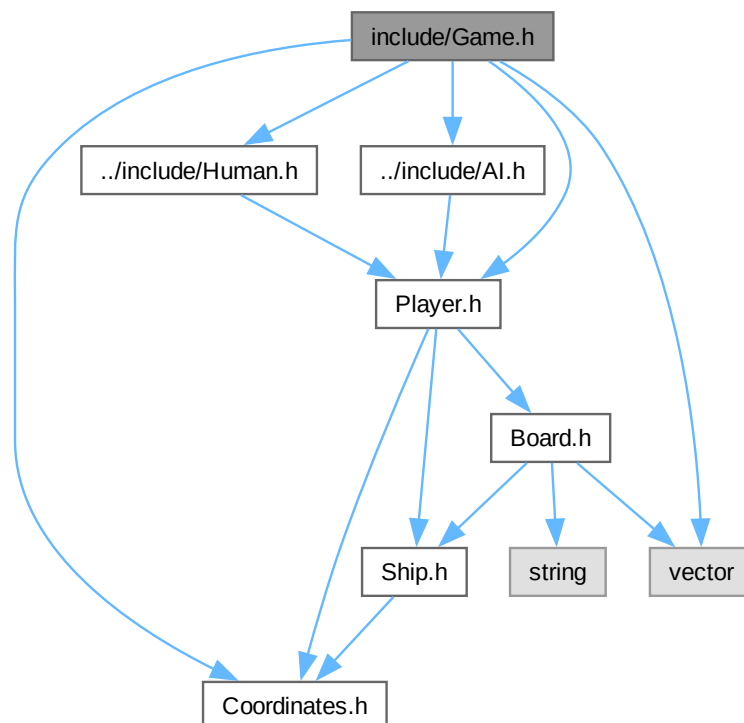
Defines The game class, which allows users to interact with the game.

```

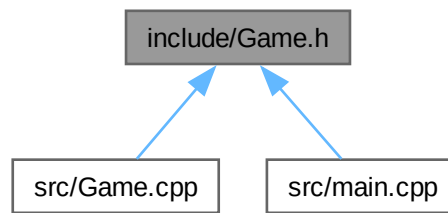
#include "../include/AI.h"
#include "../include/Coordinates.h"
#include "../include/Human.h"
#include "../include/Player.h"
#include <vector>

```

Include dependency graph for Game.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Game](#)

8.7.1 Detailed Description

Defines The game class, which allows users to interact with the game.

8.8 Game.h

[Go to the documentation of this file.](#)

```

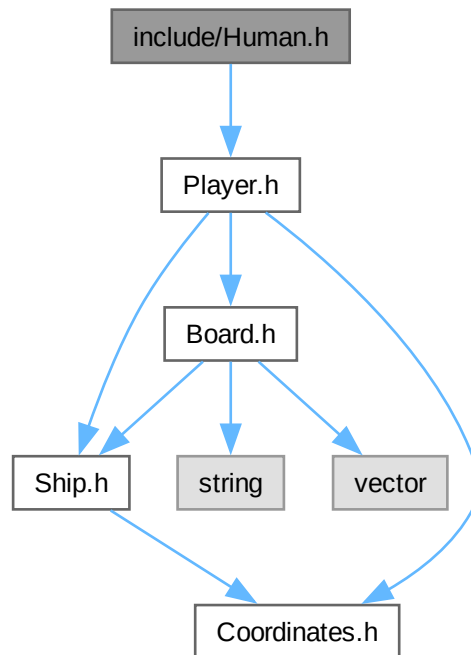
00001
00006
00007 #pragma once
00008 #include "../include/AI.h"
00009 #include "../include/Coordinates.h"
00010 #include "../include/Human.h"
00011 #include "../include/Player.h"
00012 #include <vector>
00013 /****
00014  * @class Game
00015  * @brief Implements basic game functions
00016  *
00017  */
00018 class Game {
00019 private:
00025     void printBoard(std::vector<std::vector<int>> board);
00033
00039     void printTargetBoard(std::vector<std::vector<int>> board);
00046     bool shipsAlive(std::vector<std::vector<int>> board);
00047     AI ai;
00048     Human human;
00049
00050 public:
00051     // Game();
00052     // ~Game();
00057     void runSetup();
00062     void runGame();
00063 };
  
```


8.9 include/Human.h File Reference

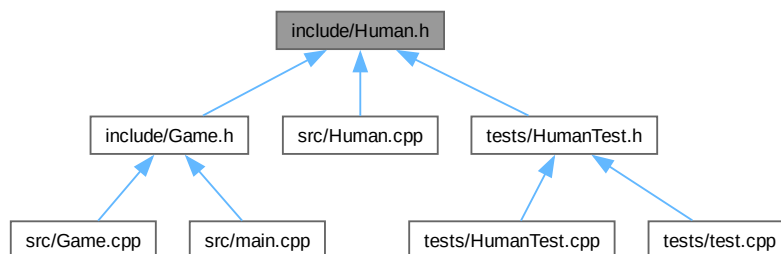
Defines the [Human](#) class, which represents a human player in a game of Battleship.

```
#include "Player.h"
```

Include dependency graph for Human.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Human](#)

Represents a human-controlled player in a game of Battleship.

8.9.1 Detailed Description

Defines the [Human](#) class, which represents a human player in a game of Battleship.

8.10 Human.h

[Go to the documentation of this file.](#)

```

00001
00006
00007 #pragma once
00008
00009 #include "Player.h"
00010
00020 class Human : public Player {
00021 public:
00025     Human();
00026
00030     ~Human() = default;
00031
00041     Coordinates fireShot() override;
00042 };

```

8.11 include/Player.h File Reference

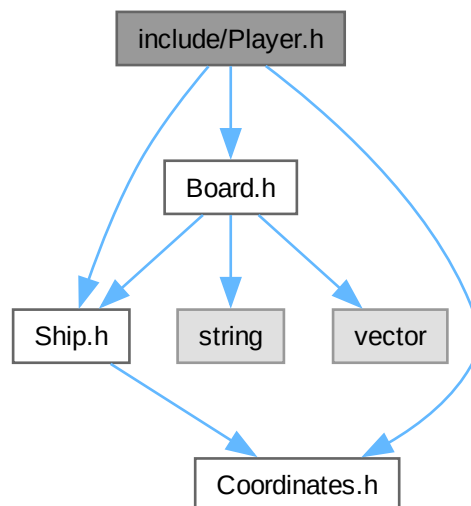
Defines the abstract [Player](#) class, which serves as the base class for [Human](#) and [AI](#) players in a game of Battleship.

```

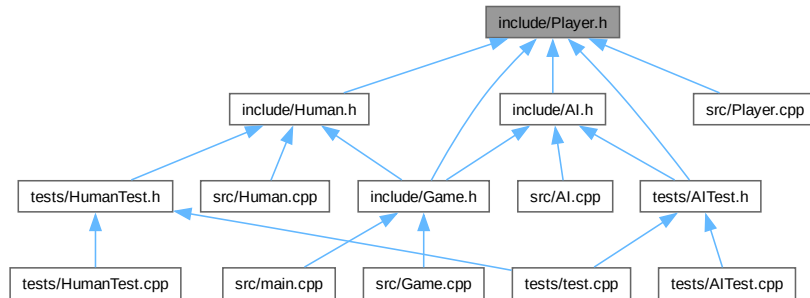
#include "Board.h"
#include "Coordinates.h"
#include "Ship.h"

```

Include dependency graph for Player.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Player](#)
Abstract base class representing a player in a game of Battleship.

8.11.1 Detailed Description

Defines the abstract [Player](#) class, which serves as the base class for [Human](#) and [AI](#) players in a game of Battleship.

8.12 Player.h

[Go to the documentation of this file.](#)

```

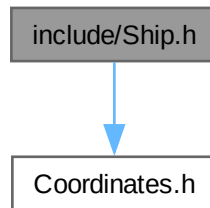
00001
00006
00007 #pragma once
00008
00009 #include "Board.h"
00010 #include "Coordinates.h"
00011 #include "Ship.h"
00012
00022 class Player {
00023 protected:
00024     Board myBoard;
00025     std::vector<std::vector<int>> targetGrid;
00028
00029 public:
00034     Player();
00035
00036     virtual ~Player() = default;
00039
00040     bool placeShip(Ship ship);
00050
00051     virtual Coordinates fireShot() = 0;
00061
00071     void updateTargetGrid(Coordinates coords, bool hit);
00072
00080     std::vector<std::vector<int>> getMyGrid() const;
00081
00089     std::vector<std::vector<int>> getTargetGrid() const;
00090
00098     bool attacked(Coordinates attack);
00099
00106     void importBoardFromFile(const std::string &filename) {
00107         myBoard.importFromFile(filename);
00108     }
00109 };
  
```

8.13 include/Ship.h File Reference

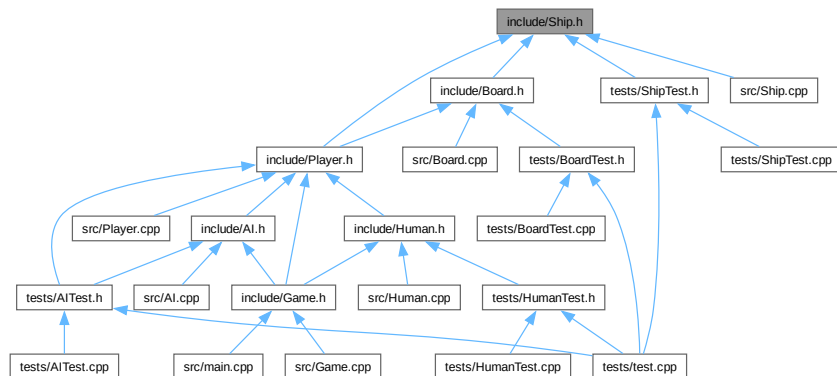
Defines the [Ship](#) class, which represents a ship in a game of Battleship.

```
#include "Coordinates.h"
```

Include dependency graph for Ship.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Ship](#)

8.13.1 Detailed Description

Defines the [Ship](#) class, which represents a ship in a game of Battleship.

8.14 Ship.h

[Go to the documentation of this file.](#)

```
00001
00006
00007 #pragma once
00008
00009 #include "Coordinates.h"
00010
00011 /****
00012  * @class Ship
00013  * @brief Represents a ship in a game of Battleship.
00014  */
00015 class Ship {
00016 private:
00017     bool isSunk;
00018     int hitCount;
00019 public:
00020     Coordinates start;
00021     Coordinates end;
00022
00026     Ship() : isSunk(false), hitCount(0), start(), end() {};
00032     ~Ship() = default;
00040     Ship(Coordinates start, Coordinates end);
00047
00048     bool checkHit(Coordinates attack);
00054     bool getIsSunk();
00055 };
```

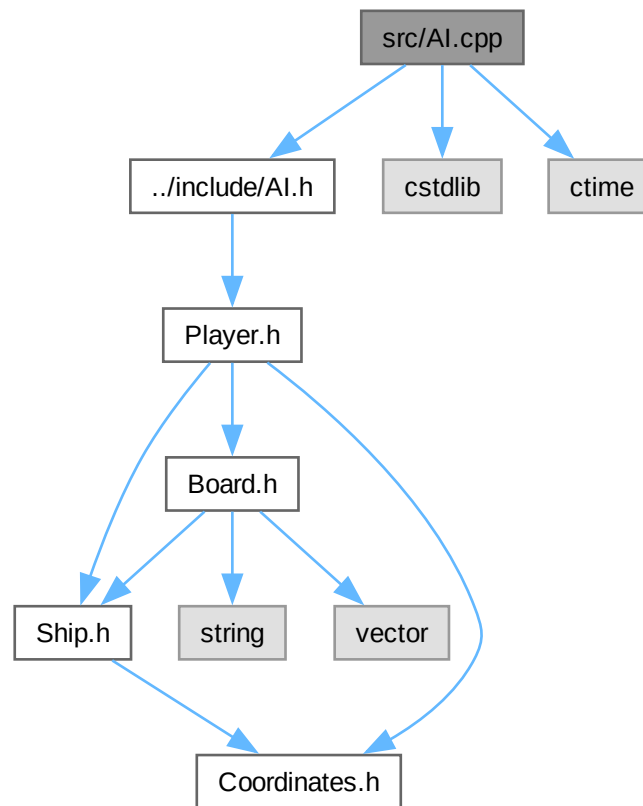
8.15 README.md File Reference

8.16 src/AI.cpp File Reference

Implements the [AI](#) class.

```
#include "../include/AI.h"
#include <cstdlib>
#include <ctime>
```

Include dependency graph for AI.cpp:



Variables

- static const int `SHIP_SIZES` [] = {5, 4, 3, 3, 2}
- static const int `SHIP_COUNT` = 5

8.16.1 Detailed Description

Implements the `AI` class.

8.16.2 Variable Documentation

8.16.2.1 SHIP_COUNT

```
const int SHIP_COUNT = 5 [static]
```

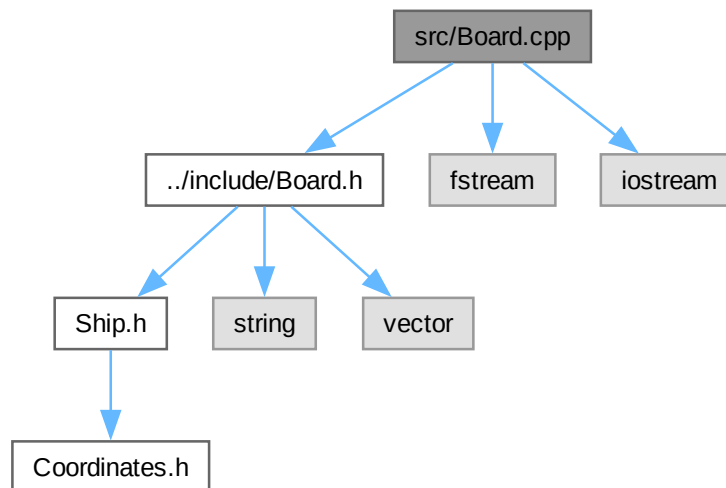
8.16.2.2 SHIP_SIZES

```
const int SHIP_SIZES[] = {5, 4, 3, 3, 2} [static]
```

8.17 src/Board.cpp File Reference

Implements the [Board](#) class, which represents the game board in a game of Battleship.

```
#include "../include/Board.h"  
#include <fstream>  
#include <iostream>  
Include dependency graph for Board.cpp:
```



8.17.1 Detailed Description

Implements the [Board](#) class, which represents the game board in a game of Battleship.

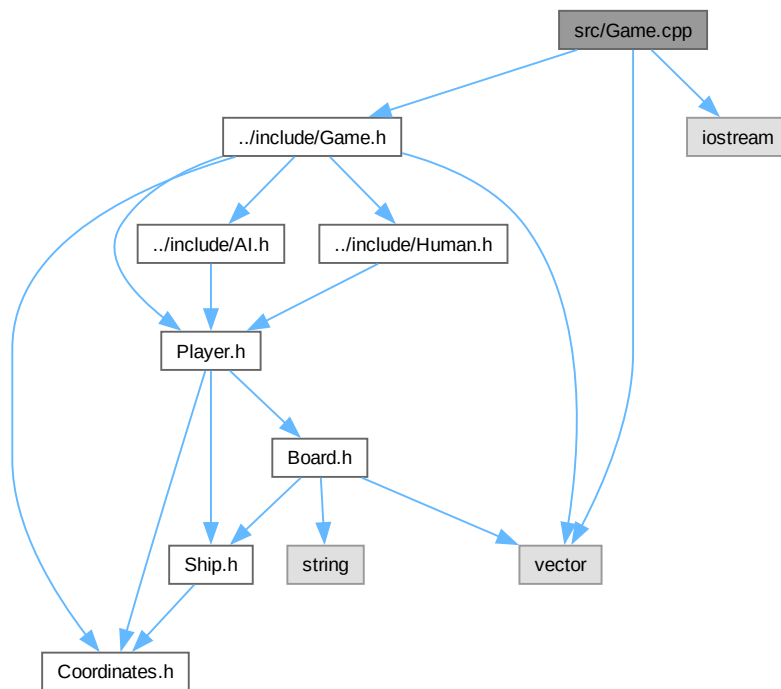
8.18 src/Game.cpp File Reference

Implementation of [Game](#) class.

```
#include "../include/Game.h"  
#include <iostream>
```

```
#include <vector>
```

Include dependency graph for Game.cpp:



8.18.1 Detailed Description

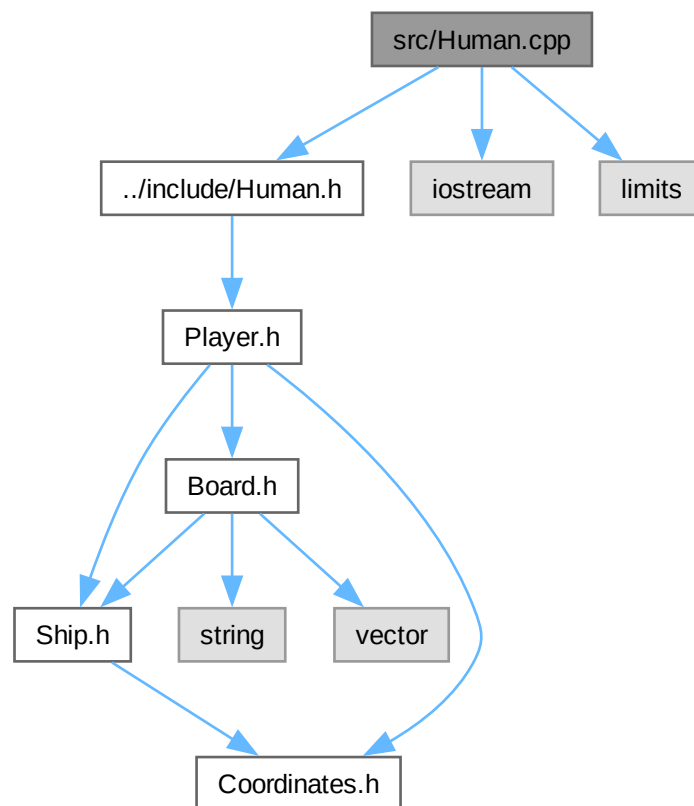
Implementation of [Game](#) class.

8.19 src/Human.cpp File Reference

Implements the [Human](#) class.

```
#include "../include/Human.h"
#include <iostream>
#include <limits>
```


Include dependency graph for Human.cpp:



8.19.1 Detailed Description

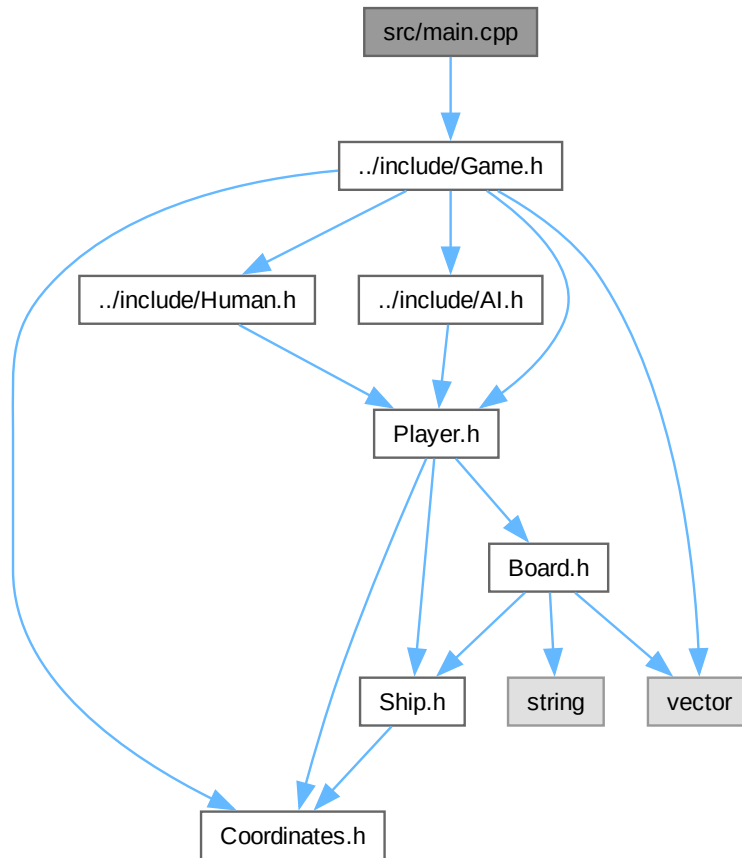
Implements the [Human](#) class.

8.20 src/main.cpp File Reference

main game file that runs the game.

```
#include "../include/Game.h"
```

Include dependency graph for main.cpp:



Functions

- int `main` ()

8.20.1 Detailed Description

main game file that runs the game.

8.20.2 Function Documentation

8.20.2.1 `main()`

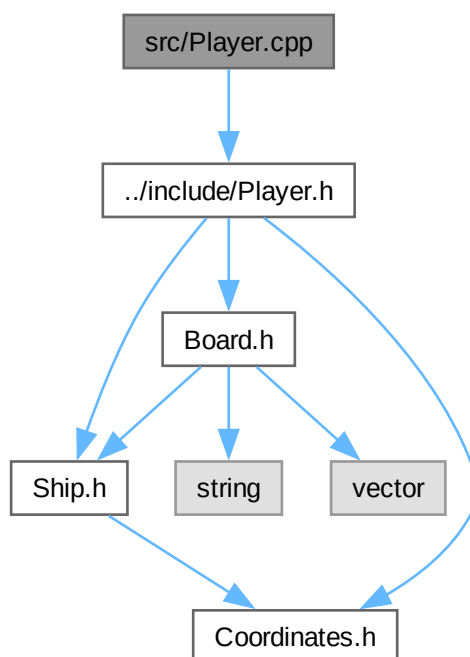
```
int main ()
```

8.21 src/Player.cpp File Reference

Implements the [Player](#) class, the abstract base for [Human](#) and [AI](#) players in a game of Battleship.

```
#include "../include/Player.h"
```

Include dependency graph for Player.cpp:



8.21.1 Detailed Description

Implements the [Player](#) class, the abstract base for [Human](#) and [AI](#) players in a game of Battleship.

8.22 src/Ship.cpp File Reference

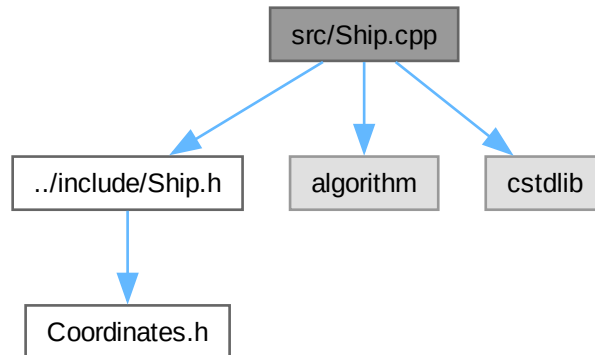
Implements the [Ship](#) class, which represents a ship in a game of Battleship.

```
#include "../include/Ship.h"
```

```
#include <algorithm>
```

```
#include <cstdlib>
```

Include dependency graph for Ship.cpp:



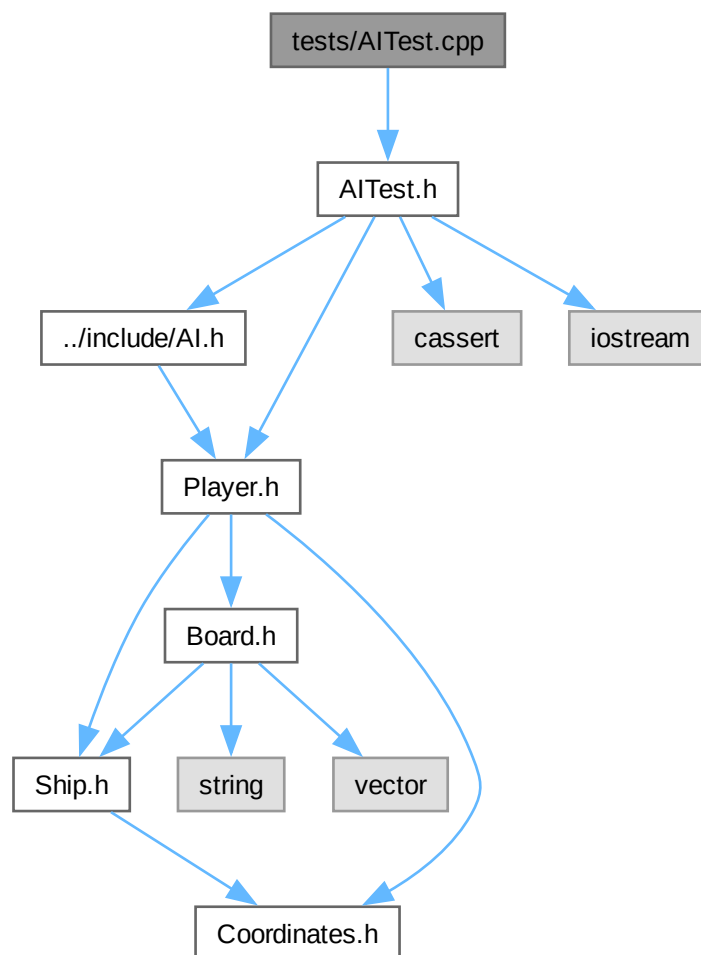
8.22.1 Detailed Description

Implements the [Ship](#) class, which represents a ship in a game of Battleship.

8.23 tests/AITest.cpp File Reference

```
#include "AITest.h"
```

Include dependency graph for AITest.cpp:



Functions

- void `testAI` ()

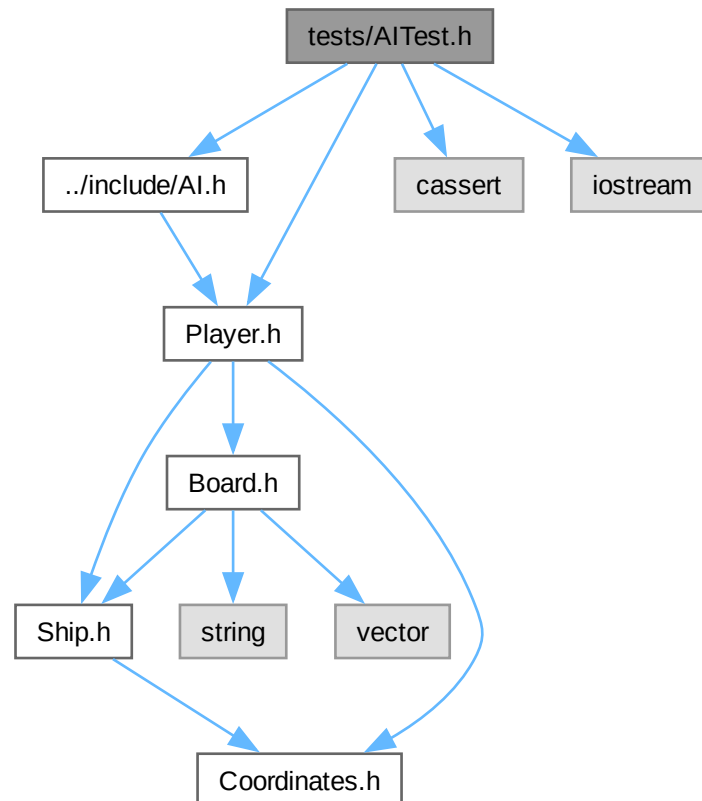
8.23.1 Function Documentation

8.23.1.1 testAI()

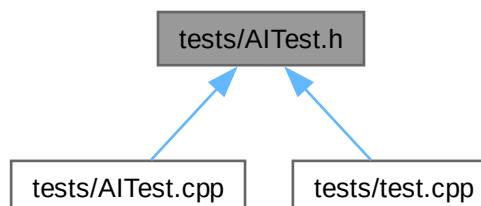
```
void testAI ()
```

8.24 tests/AITest.h File Reference

```
#include "../include/AI.h"  
#include "../include/Player.h"  
#include <cassert>  
#include <iostream>  
Include dependency graph for AITest.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void `testAI` ()

8.24.1 Function Documentation

8.24.1.1 testAI()

```
void testAI ()
```

8.25 AITest.h

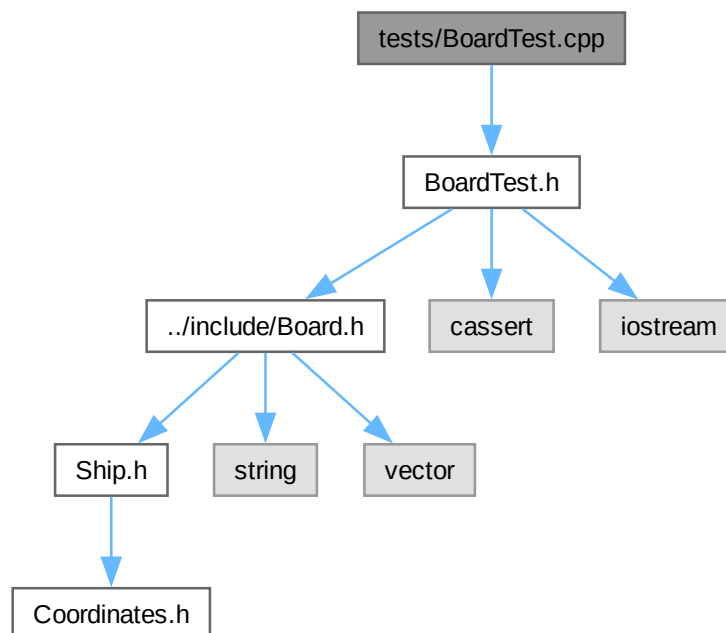
[Go to the documentation of this file.](#)

```
00001 #include "../include/AI.h"
00002 #include "../include/Player.h"
00003 #include <cassert>
00004 #include <iostream>
00005
00006 void testAI();
```

8.26 tests/BoardTest.cpp File Reference

```
#include "BoardTest.h"
```

Include dependency graph for BoardTest.cpp:



Functions

- void `testBoard` ()

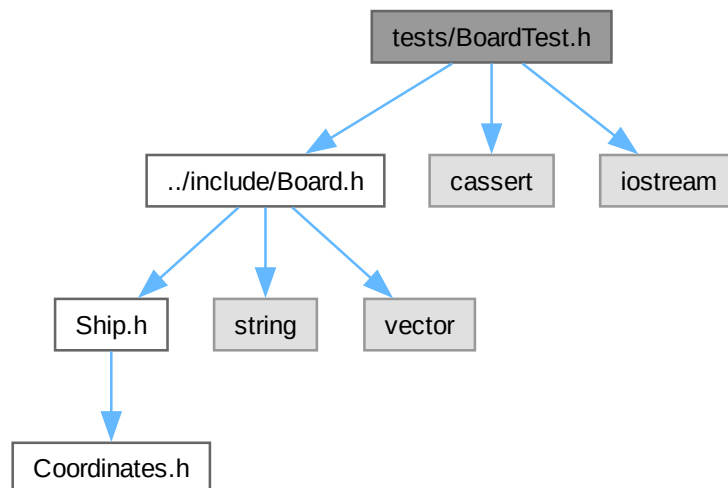
8.26.1 Function Documentation

8.26.1.1 `testBoard()`

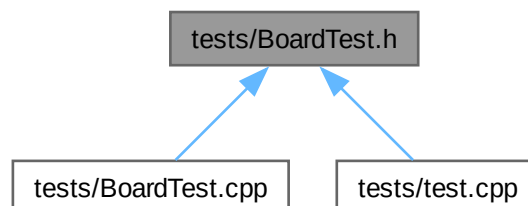
```
void testBoard ()
```

8.27 `tests/BoardTest.h` File Reference

```
#include "../include/Board.h"  
#include <cassert>  
#include <iostream>  
Include dependency graph for BoardTest.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void `testBoard` ()

8.27.1 Function Documentation

8.27.1.1 `testBoard()`

```
void testBoard ()
```

8.28 BoardTest.h

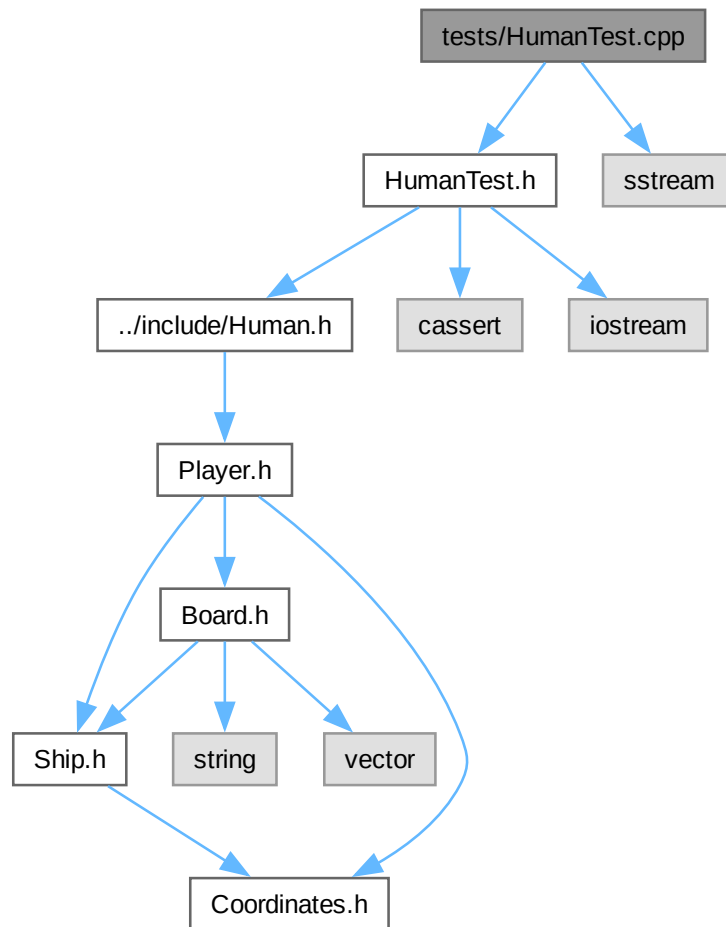
[Go to the documentation of this file.](#)

```
00001 #include "../include/Board.h"
00002 #include <cassert>
00003 #include <iostream>
00004
00005 void testBoard();
```

8.29 tests/HumanTest.cpp File Reference

```
#include "HumanTest.h"
#include <sstream>
```

Include dependency graph for HumanTest.cpp:



Functions

- void `testHuman()`

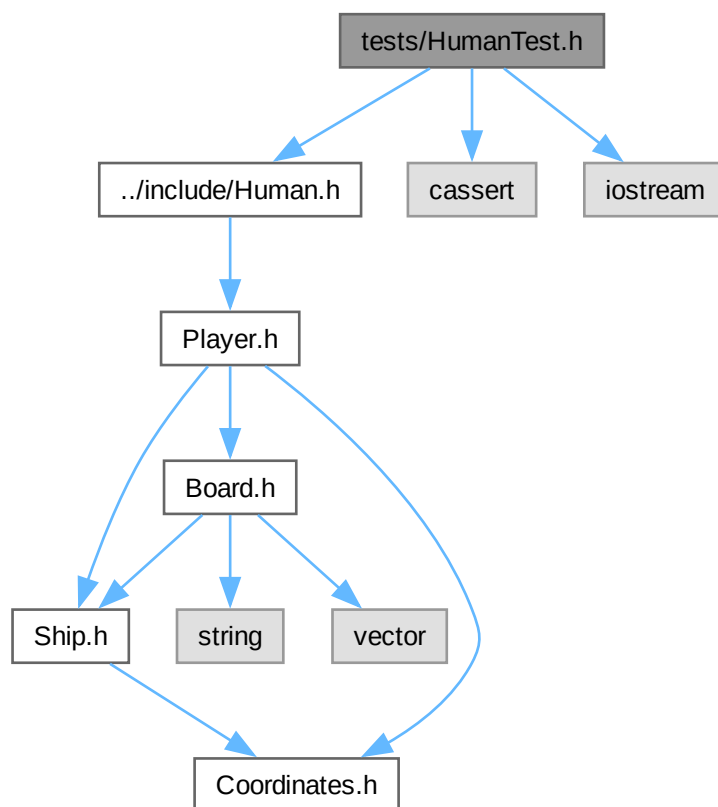
8.29.1 Function Documentation

8.29.1.1 `testHuman()`

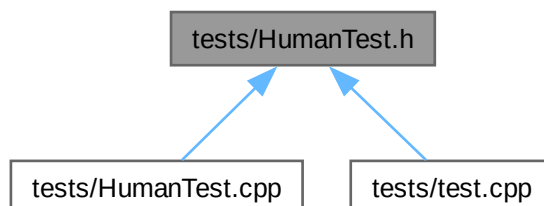
```
void testHuman ()
```

8.30 tests/HumanTest.h File Reference

```
#include "../include/Human.h"  
#include <cassert>  
#include <iostream>  
Include dependency graph for HumanTest.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- void [testHuman](#) ()

8.30.1 Function Documentation

8.30.1.1 testHuman()

```
void testHuman ()
```

8.31 HumanTest.h

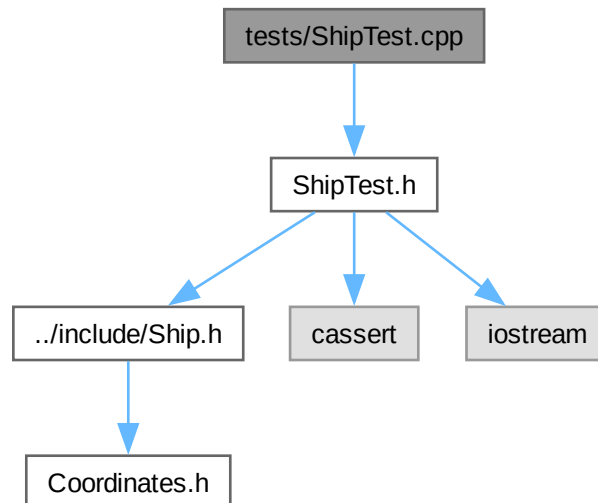
[Go to the documentation of this file.](#)

```
00001 #include "../include/Human.h"
00002 #include <cassert>
00003 #include <iostream>
00004
00005 void testHuman();
```

8.32 tests/ShipTest.cpp File Reference

```
#include "ShipTest.h"
```

Include dependency graph for ShipTest.cpp:



Functions

- void [testShip](#) ()

8.32.1 Function Documentation

8.32.1.1 testShip()

```
void testShip ()
```

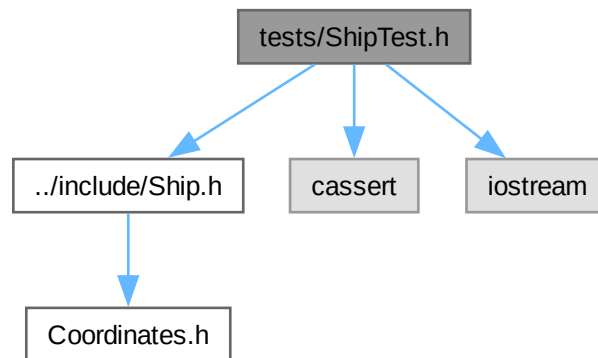
8.33 tests/ShipTest.h File Reference

```
#include "../include/Ship.h"
```

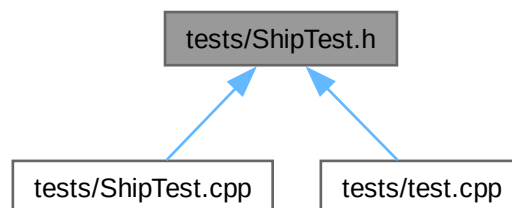
```
#include <cassert>
```

```
#include <iostream>
```

Include dependency graph for ShipTest.h:



This graph shows which files directly or indirectly include this file:



Functions

- void `testShip` ()

8.33.1 Function Documentation

8.33.1.1 testShip()

```
void testShip ()
```

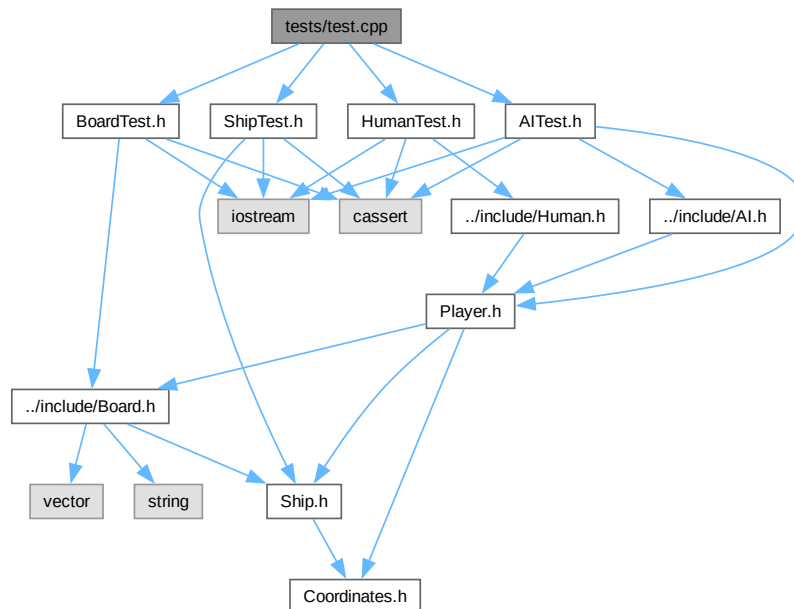
8.34 ShipTest.h

[Go to the documentation of this file.](#)

```
00001 #include "../include/Ship.h"
00002 #include <cassert>
00003 #include <iostream>
00004
00005 void testShip();
```

8.35 tests/test.cpp File Reference

```
#include "BoardTest.h"
#include "ShipTest.h"
#include "HumanTest.h"
#include "AITest.h"
Include dependency graph for test.cpp:
```



Functions

- int [main](#) ()

8.35.1 Function Documentation

8.35.1.1 main()

```
int main ()
```


Index

- ~AI
 - AI, [15](#)
- ~Board
 - Board, [19](#)
- ~Human
 - Human, [29](#)
- ~Player
 - Player, [35](#)
- ~Ship
 - Ship, [40](#)
- AI, [13](#)
 - ~AI, [15](#)
 - AI, [15](#)
 - attacked, [16](#)
 - fireShot, [16](#)
 - getMyGrid, [16](#)
 - getTargetGrid, [16](#)
 - importBoardFromFile, [17](#)
 - myBoard, [18](#)
 - placeShip, [17](#)
 - setupGrid, [17](#)
 - targetGrid, [18](#)
 - updateTargetGrid, [17](#)
- ai
 - Game, [26](#)
- AI.cpp
 - SHIP_COUNT, [54](#)
 - SHIP_SIZES, [54](#)
- AITest.cpp
 - testAI, [61](#)
- AITest.h
 - testAI, [63](#)
- attack
 - Board, [20](#)
- attacked
 - AI, [16](#)
 - Human, [30](#)
 - Player, [36](#)
- Battleship, [1](#)
- Board, [18](#)
 - ~Board, [19](#)
 - attack, [20](#)
 - Board, [19](#)
 - getGrid, [20](#)
 - grid, [21](#)
 - importFromFile, [20](#)
 - placeShip, [21](#)
 - ships, [21](#)
- BoardTest.cpp
 - testBoard, [64](#)
- BoardTest.h
 - testBoard, [65](#)
- checkHit
 - Ship, [40](#)
- Coordinates, [21](#)
 - Coordinates, [22](#)
 - x, [22](#)
 - y, [22](#)
- end
 - Ship, [41](#)
- fireShot
 - AI, [16](#)
 - Human, [30](#)
 - Player, [36](#)
- Game, [23](#)
 - ai, [26](#)
 - human, [26](#)
 - printBoard, [25](#)
 - printTargetBoard, [25](#)
 - runGame, [26](#)
 - runSetup, [26](#)
 - shipsAlive, [26](#)
- getGrid
 - Board, [20](#)
- getIsSunk
 - Ship, [40](#)
- getMyGrid
 - AI, [16](#)
 - Human, [30](#)
 - Player, [36](#)
- getTargetGrid
 - AI, [16](#)
 - Human, [30](#)
 - Player, [36](#)
- grid
 - Board, [21](#)
- hitCount
 - Ship, [41](#)
- Human, [27](#)
 - ~Human, [29](#)
 - attacked, [30](#)
 - fireShot, [30](#)
 - getMyGrid, [30](#)
 - getTargetGrid, [30](#)

- Human, 29
- importBoardFromFile, 31
- myBoard, 32
- placeShip, 31
- targetGrid, 32
- updateTargetGrid, 31
- human
 - Game, 26
- HumanTest.cpp
 - testHuman, 66
- HumanTest.h
 - testHuman, 68
- importBoardFromFile
 - AI, 17
 - Human, 31
 - Player, 37
- importFromFile
 - Board, 20
- include Directory Reference, 11
- include/AI.h, 43, 44
- include/Board.h, 44, 45
- include/Coordinates.h, 46, 47
- include/Game.h, 47, 48
- include/Human.h, 49, 50
- include/Player.h, 50, 51
- include/Ship.h, 52, 53
- isSunk
 - Ship, 41
- main
 - main.cpp, 58
 - test.cpp, 71
- main.cpp
 - main, 58
- myBoard
 - AI, 18
 - Human, 32
 - Player, 38
- placeShip
 - AI, 17
 - Board, 21
 - Human, 31
 - Player, 37
- Player, 32
 - ~Player, 35
 - attacked, 36
 - fireShot, 36
 - getMyGrid, 36
 - getTargetGrid, 36
 - importBoardFromFile, 37
 - myBoard, 38
 - placeShip, 37
 - Player, 35
 - targetGrid, 38
 - updateTargetGrid, 37
- printBoard
 - Game, 25
- printTargetBoard
 - Game, 25
- README.md, 53
- runGame
 - Game, 26
- runSetup
 - Game, 26
- setupGrid
 - AI, 17
- Ship, 38
 - ~Ship, 40
 - checkHit, 40
 - end, 41
 - getIsSunk, 40
 - hitCount, 41
 - isSunk, 41
 - Ship, 40
 - start, 41
- SHIP_COUNT
 - AI.cpp, 54
- SHIP_SIZES
 - AI.cpp, 54
- ships
 - Board, 21
- shipsAlive
 - Game, 26
- ShipTest.cpp
 - testShip, 69
- ShipTest.h
 - testShip, 70
- src Directory Reference, 11
- src/AI.cpp, 53
- src/Board.cpp, 55
- src/Game.cpp, 55
- src/Human.cpp, 56
- src/main.cpp, 57
- src/Player.cpp, 59
- src/Ship.cpp, 59
- start
 - Ship, 41
- targetGrid
 - AI, 18
 - Human, 32
 - Player, 38
- test.cpp
 - main, 71
- testAI
 - AITest.cpp, 61
 - AITest.h, 63
- testBoard
 - BoardTest.cpp, 64
 - BoardTest.h, 65
- testHuman
 - HumanTest.cpp, 66
 - HumanTest.h, 68
- tests Directory Reference, 12

- tests/AlTest.cpp, [61](#)
- tests/AlTest.h, [62](#), [63](#)
- tests/BoardTest.cpp, [63](#)
- tests/BoardTest.h, [64](#), [65](#)
- tests/HumanTest.cpp, [65](#)
- tests/HumanTest.h, [67](#), [68](#)
- tests/ShipTest.cpp, [68](#)
- tests/ShipTest.h, [69](#), [70](#)
- tests/test.cpp, [70](#)
- testShip
 - ShipTest.cpp, [69](#)
 - ShipTest.h, [70](#)
- updateTargetGrid
 - AI, [17](#)
 - Human, [31](#)
 - Player, [37](#)
- x
 - Coordinates, [22](#)
- y
 - Coordinates, [22](#)